

EXPERIMENT 3

GREEDY ALGORITHMS: KNAPSACK AND ACTIVITY SELECTION

Common Aim: To implement algorithms based on the greedy approach to solve the knapsack problem and the activity selection problem.

Language: Java **Compiler:** javac 26.0.1 **Runtime:** Java HotSpot 64-Bit Server VM (26.0.1+8-34)

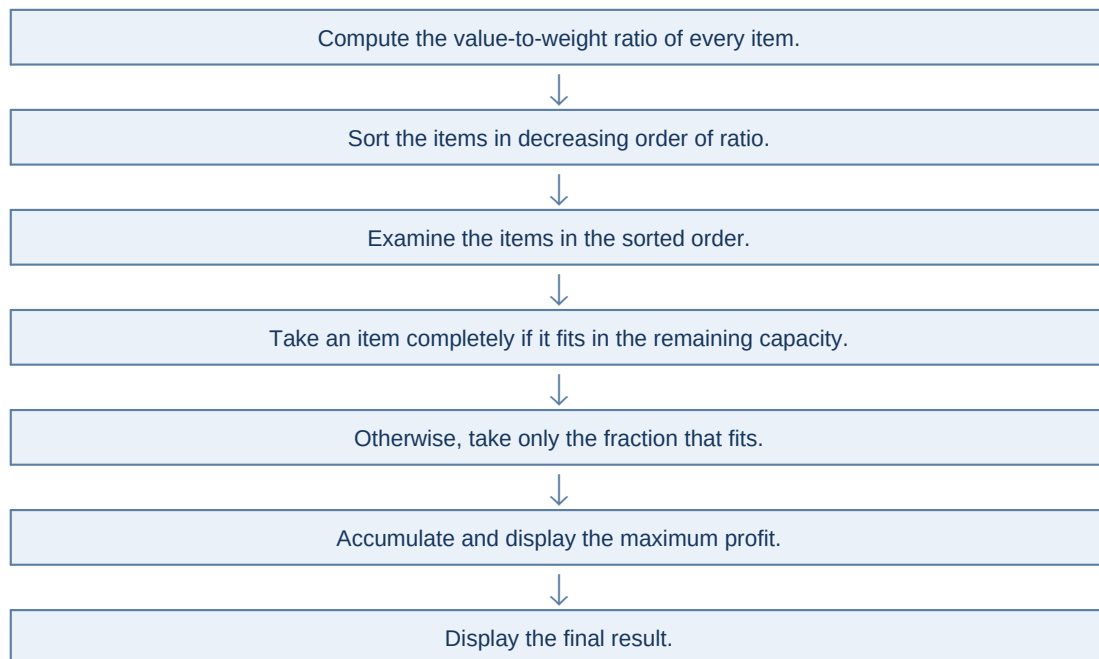
3.a Fractional Knapsack

Aim: To fill a knapsack with items that may be divided into fractions so that the total profit is maximized. The greedy method selects items in decreasing order of value-to-weight ratio.

Algorithm:

1. Compute the value-to-weight ratio of every item.
2. Sort the items in decreasing order of ratio.
3. Examine the items in the sorted order.
4. Take an item completely if it fits in the remaining capacity.
5. Otherwise, take only the fraction that fits.
6. Accumulate and display the maximum profit.

Represented Workflow:



Program:

```
import java.util.*;
public class FractionalKnapsack {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of items: ");
        int n = sc.nextInt();
        double[][] it = new double[n][4];          // weight, value, ratio, id
        System.out.println("Enter the weight and value of each item: ");
        for (int i = 0; i < n; i++) {
```

```

        it[i][0] = sc.nextDouble();    it[i][1] = sc.nextDouble();
        it[i][2] = it[i][1] / it[i][0];    it[i][3] = i + 1;
    }
    System.out.print("Enter the capacity of the knapsack: ");
    double cap = sc.nextDouble();
    Arrays.sort(it, (a, b) -> Double.compare(b[2], a[2]));
    double profit = 0;
    for (int i = 0; i < n && cap > 0; i++) {
        double take = Math.min(it[i][0], cap);
        profit += take * it[i][2];    cap -= take;
        System.out.printf("Item %d taken with fraction %.2f%n", (int) it[i][3], take / it[i][0]);
    }
    System.out.printf("Maximum profit is %.2f%n", profit);
}
}

```

Output:

```

Enter the number of items: 3
Enter the weight and value of each item:
10 60
20 100
30 120
Enter the capacity of the knapsack: 50
Item 1 taken with fraction 1.00
Item 2 taken with fraction 1.00
Item 3 taken with fraction 0.67
Maximum profit is 240.00

```

Result: Thus, the fractional knapsack problem was implemented successfully using the greedy approach. The tested output produced a maximum profit of 240.00.

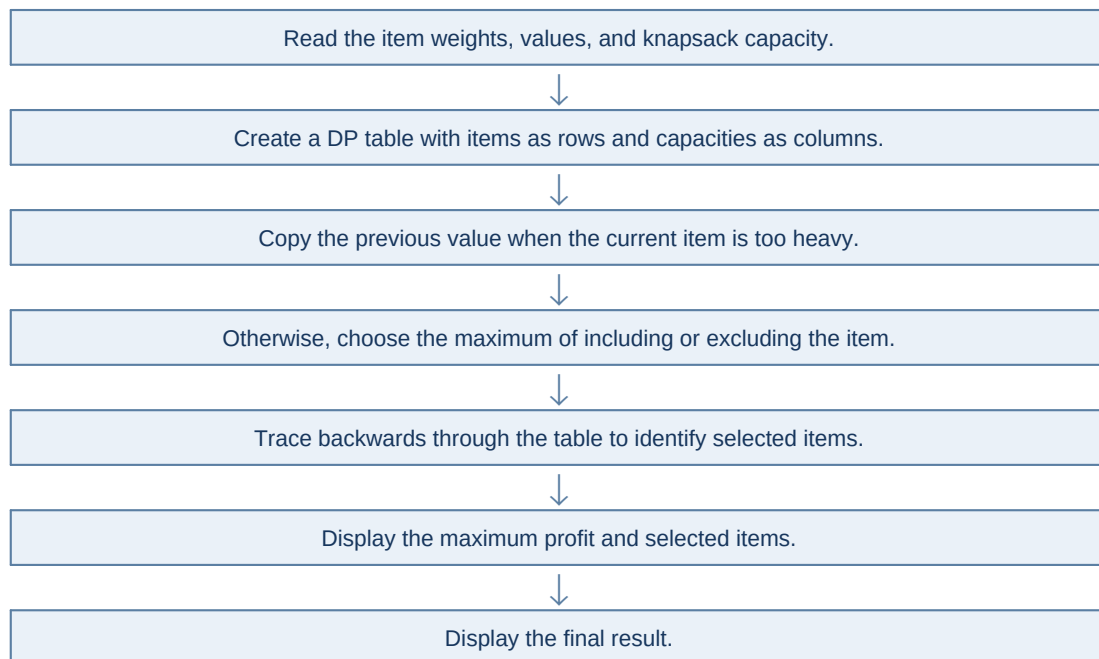
3.b 0/1 Knapsack

Aim: To fill a knapsack with whole items so that the total profit is maximized. Dynamic programming is used because fractional greedy selection does not always give the optimum for indivisible items.

Algorithm:

1. Read the item weights, values, and knapsack capacity.
2. Create a DP table with items as rows and capacities as columns.
3. Copy the previous value when the current item is too heavy.
4. Otherwise, choose the maximum of including or excluding the item.
5. Trace backwards through the table to identify selected items.
6. Display the maximum profit and selected items.

Represented Workflow:



Program:

```
import java.util.*;
public class Knapsack01 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of items: ");
        int n = sc.nextInt();
        int[] w = new int[n + 1], v = new int[n + 1];
        System.out.println("Enter the weight and value of each item: ");
        for (int i = 1; i <= n; i++) { w[i] = sc.nextInt(); v[i] = sc.nextInt(); }
        System.out.print("Enter the capacity of the knapsack: ");
        int cap = sc.nextInt();
        int[][] dp = new int[n + 1][cap + 1];
        for (int i = 1; i <= n; i++)
            for (int c = 0; c <= cap; c++)
                dp[i][c] = (w[i] <= c) ? Math.max(dp[i-1][c], dp[i-1][c-w[i]] + v[i]) : dp[i-1][c];
        System.out.println("Maximum profit is " + dp[n][cap]);
        System.out.print("Items selected: ");
        for (int i = n, c = cap; i > 0; i--)
            if (dp[i][c] != dp[i-1][c]) { System.out.print(i + " "); c -= w[i]; }
        System.out.println();
    }
}
```

Output:

Enter the number of items: 3

```
Enter the weight and value of each item:  
10 60  
20 100  
30 120  
Enter the capacity of the knapsack: 50  
Maximum profit is 220  
Items selected: 3 2
```

Result: Thus, the 0/1 knapsack problem was implemented successfully using dynamic programming. The tested output produced an optimal profit of 220 using items 3 and 2.

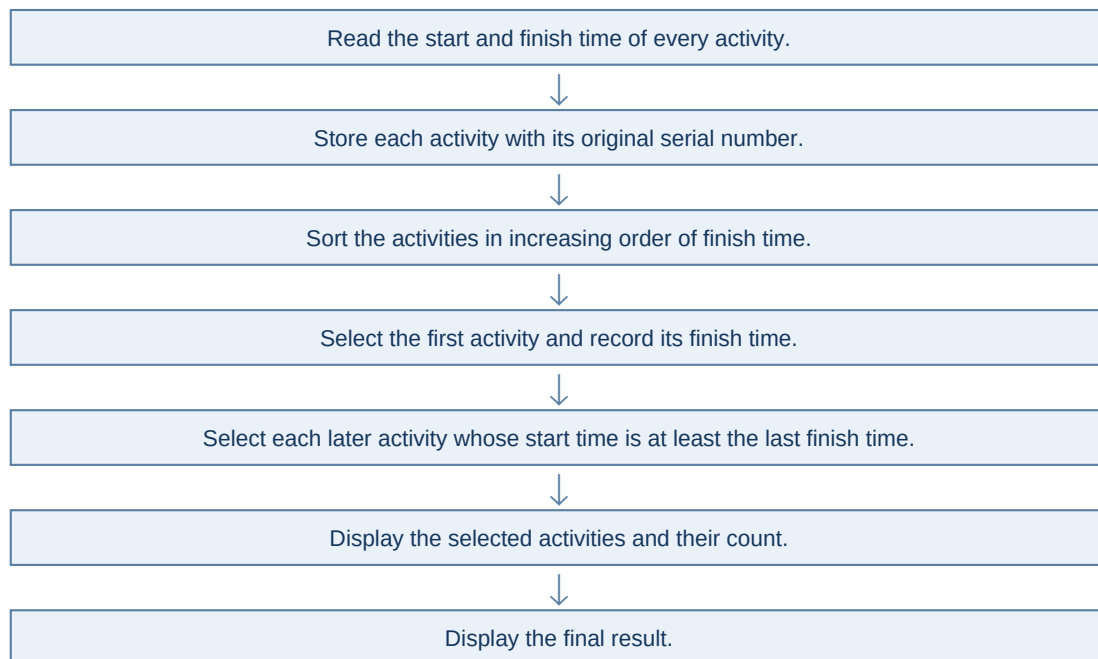
3.c Activity Selection

Aim: To select the maximum number of mutually compatible activities from their start and finish times. The greedy method repeatedly chooses the compatible activity that finishes earliest.

Algorithm:

1. Read the start and finish time of every activity.
2. Store each activity with its original serial number.
3. Sort the activities in increasing order of finish time.
4. Select the first activity and record its finish time.
5. Select each later activity whose start time is at least the last finish time.
6. Display the selected activities and their count.

Represented Workflow:



Program:

```
import java.util.*;
public class ActivitySelection {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of activities: ");
        int n = sc.nextInt();
        int[][] a = new int[n][3]; // start, finish, id
        System.out.println("Enter the start and finish time of each activity: ");
        for (int i = 0; i < n; i++) { a[i][0] = sc.nextInt(); a[i][1] = sc.nextInt(); a[i][2] = i + 1; }
        Arrays.sort(a, (x, y) -> x[1] - y[1]);
        int count = 0, last = Integer.MIN_VALUE;
        System.out.print("Selected activities: ");
        for (int i = 0; i < n; i++)
            if (a[i][0] >= last) {
                System.out.print(a[i][2] + "(" + a[i][0] + "," + a[i][1] + ") ");
                last = a[i][1]; count++;
            }
        System.out.println("\nMaximum number of activities is " + count);
    }
}
```

Output:

```
Enter the number of activities: 6
Enter the start and finish time of each activity:
```

```
1 2
3 4
0 6
5 7
8 9
5 9
```

Selected activities: 1(1,2) 2(3,4) 4(5,7) 5(8,9)

Maximum number of activities is 4

Result: Thus, the activity selection problem was implemented successfully using the greedy approach. The tested output selected four mutually compatible activities.